

Production Capability List Divergence Audit

Production GET /enterprise/options returns 7 capability codes; the canonical allow-list defines 12. Five codes are missing on production. This report determines why, by reviewing every filter the code applies and cross-checking the live dev DB. No code modified.

ROOT CAUSE — The code is deterministic and behaves identically across environments. Dev returns all 12 canonical codes; production returns 7. The only filter capable of excluding the 5 missing codes is the `isActive: true` clause in `getOptions()`'s `prisma.capability.findMany`. The missing rows are therefore either inactive (`isActive=false`) or absent on production. The catalogue page (/capabilities) supports row deactivation but not deletion, so the most probable cause is an admin deactivation. Confirmation requires a direct probe of production (the connected dev DB has the same code path and returns all 12, so the divergence is data, not code).

1. Root cause

The committed code path — `canonical-capabilities.ts`, `enterprise.service.ts.getOptions()`, `capability-groups.ts` — is deterministic. Same inputs produce the same output. Given:

- `CANONICAL_CAPABILITY_CODES` at `canonical-capabilities.ts:14-30` includes all 12 codes (INSPECTION, MAINTENANCE, QA_VALIDATION, REPORTING, SAVR, SAVT, PENCAWANG, FEEDER_PILLAR, LINK_BOX, CABLE_BRIDGE, UNDERGROUND_CABLE, THERMAL_INSPECTION). Verified by re-read.
- The only server-side filter on the capabilities list is the `where` clause at `enterprise.service.ts:437-440`: `{ isActive: true, code: { in: Array.from(CANONICAL_CAPABILITY_CODES) } }`. No tenant filter, no role filter, no environment-specific filter.
- `capability-groups.ts` is client-side and cannot expand the API response; it only further filters / groups what the API returned.
- The connected dev DB returns 12 codes through this exact query (verified at 2026-06-03 by replicating the query). Production returns 7 through the same code. Therefore the divergence is in the data the query reads, not in the code.
- A canonical-list capability row is dropped from the response if and only if its `Capability.isActive` is false, OR the row does not exist. There is no third exclusion path.

On production, the five missing codes — INSPECTION, MAINTENANCE, REPORTING, THERMAL_INSPECTION, UNDERGROUND_CABLE — must therefore satisfy one of those two conditions. Most probably `isActive=false`, because the deployed admin endpoints offer a deactivation toggle on the catalogue page but no delete endpoint (PATCH /enterprise/capabilities/:id/status exists; DELETE does not).

2. Do the missing capabilities exist in the database? 3. Are they inactive?

Cannot be determined from the source repository alone — these are runtime data questions and require a probe against production. On the connected dev DB:

Code	Dev: exists?	Dev: isActive	Dev: assignments (org/branch/mh/team/user, active only)
INSPECTION	Yes (id ...011)	true	2 / 1 / 1 / 1 / 4
MAINTENANCE	Yes (id ...009)	true	1 / 1 / 0 / 0 / 1
REPORTING	Yes (id ...018)	true	2 / 0 / 0 / 0 / 1
THERMAL_INSPECTION	Yes (id ...008)	true	0 / 0 / 0 / 0 / 0
UNDERGROUND_CABLE	Yes (id ...007)	true	0 / 0 / 0 / 0 / 0

All five rows exist on dev with stable migration-seeded IDs (allocated in the Sprint G1 migration 20260521113000, lines 236, 237, 238, 240, 247) and are `isActive=true`. On dev, the production-observed response of 7 codes is impossible to reproduce through this code.

To determine the production state

Run the following Prisma probe against production (DATABASE_URL pointed at prod). Read-only; no writes:

```
prisma.capability.findMany({
  where: { code: { in: ['INSPECTION', 'MAINTENANCE', 'REPORTING',
    'THERMAL_INSPECTION', 'UNDERGROUND_CABLE'] } },
  select: { id: true, code: true, name: true, isActive: true,
    createdAt: true, updatedAt: true },
  orderBy: { code: 'asc' },
})
```

- If the result has 5 rows, all `isActive=false`: rows exist but were deactivated. **This is the most probable case** given the absence of a delete endpoint in the catalogue UI.
- If the result has fewer than 5 rows: those rows were deleted post-migration (only possible via direct SQL — no app endpoint deletes capabilities) or the production database never received the Sprint G1 migration that seeded them.
- If `updatedAt` on the inactive rows is later than `createdAt`, the deactivation event is recoverable from the audit log around that timestamp.

4. Is filtering logic excluding them?

Yes — specifically the `isActive: true` clause in `getOptions()`. The code `IN (...)` clause does NOT exclude them — all five missing codes are members of `CANONICAL_CAPABILITY_CODES`. The `isActive: true` clause is the only gate that can drop a row that is otherwise canonical.

Filter audit, top-to-bottom:

Filter	Where	Drops the 5 missing codes?
<code>code: { in: [...CANONICAL_CAPABILITY_CODES] }</code>	<code>enterprise.service.ts:439</code>	No. All five codes are in the canonical set.
<code>isActive: true</code>	<code>enterprise.service.ts:438</code>	Yes — if those rows are inactive. This is the only gate that can exclude a canonical row.
Client-side <code>isAssignableCapability</code>	<code>capability-groups.ts:83-91</code>	No. Cannot drop rows the API did not return. It accepts canonical + active and would happily render the missing codes if the API returned them.
Client-side <code>groupCapabilities</code>	<code>capability-groups.ts:152-186</code>	Indirect. It drops empty group buckets — so if the API returns zero workspace codes, the entire Workspace Access fieldset is suppressed. This is what the User form is currently doing, but it's a downstream effect, not the cause.
Server-side <code>MAINHEAD_PICKER_GROUP_KEYS</code>	<code>capability-groups.ts:143</code>	Not applicable. Used only by the MAINHEAD modal (<code>enterprise-list-client.tsx:858</code>), not by the User form and not by the API.

5. Exact file and code path responsible

File: `apps/api/src/enterprise/enterprise.service.ts`

Function: `EnterpriseService.getOptions()`

Lines: 436–449

```
this.prisma.capability.findMany({
  where: {
    isActive: true,      // ← the gate that excludes inactive canonical rows
    code: { in: Array.from(CANONICAL_CAPABILITY_CODES) },
  },
  orderBy: [
    { isActive: 'desc' },
    { name: 'asc' },
  ],
  select: { id, name, code, description, isActive },
})
```

The clause `code: { in: Array.from(CANONICAL_CAPABILITY_CODES) }` is sourced from `apps/api/src/common/canonical-capabilities.ts:14-30`. That constant is correct and unchanged across all environments built from this commit. `capability-groups.ts` is purely client-side and cannot affect the API response shape. `getOptions()` has no other filter on the capabilities array.

6. Sanity check on the production response

The production response (7 codes, alphabetised) is consistent with the committed query — same ordering (`{ isActive: 'desc' }, { name: 'asc' }`):

Code	Display name	In canonical set?
CABLE_BRIDGE	Cable Bridge	Yes (Asset Domain)
FEEDER_PILLAR	Feeder Pillar	Yes (Asset Domain)
LINK_BOX	Link Box	Yes (Asset Domain)
PENCAWANG	Pencawang	Yes (Asset Domain)
QA_VALIDATION	QA Validation	Yes (Governance & Reporting)
SAVR	SAVR	Yes (Asset Domain)
SAVT	SAVT	Yes (Asset Domain)

All seven codes are canonical. None of the seven legacy codes (SURVEY / REPAIR / CIVIL / DISTRIBUTION / THIRTY_THREE_KV / EMERGENCY_RESPONSE / OTHER) appear, which confirms the canonical filter **is** being applied — ruling out the possibility that production is running a pre-allow-list build of the API.

7. Why the User form lost the Workspace Access fieldset

`groupCapabilities()` (`capability-groups.ts:170–185`) drops empty group buckets. With both INSPECTION and MAINTENANCE absent from the API response, the WORKSPACE bucket contains zero capabilities, the helper returns no group entry, and the fieldset's `<legend>` never renders. The same mechanism would silently drop the Governance & Reporting fieldset if REPORTING and QA_VALIDATION both disappeared; today only REPORTING is missing, so the Governance group still has one item and remains visible.

This is consistent with the symptom the user reported: Workspace Access not shown; Governance & Reporting still shown (only QA Validation visible there); Asset Domains still shown (with the 6 present Asset Domain codes, minus UNDERGROUND_CABLE and THERMAL_INSPECTION).

8. Recommended next checks on production

- Run the read-only Prisma probe in §3 against production. Determines, per row, whether the missing codes are inactive or absent.
- If inactive: check audit logs around each row's `updatedAt` timestamp to identify who toggled the row off. `PATCH /enterprise/capabilities/:id/status` is the only app endpoint that flips this flag.
- If absent: confirm the Sprint G1 migration (20260521113000) was applied to the production DB. `SELECT * FROM "_prisma_migrations" WHERE "migration_name" LIKE '%sprint_g1%'`; The migration includes the `ON CONFLICT ("code") DO NOTHING` clause, so it can be re-run safely against a partial state.
- After the cause is established, reactivation is a single `PATCH /enterprise/capabilities/:id/status` per row (via the existing Capabilities admin page). Any pre-existing inactive `*Capability` assignment rows will become active again because the resolver's `capability: { isActive: true }` nested filter is satisfied once the row is reactivated. No data restoration required if the rows still exist.

- If the rows are absent, re-seeding via the migration's `INSERT ... ON CONFLICT DO NOTHING` recreates them with the fixed UUIDs (100000000-0000-4000-8000-...007/...008/...009/...011/...018). Existing assignment rows that reference those UUIDs would also revive.

9. Bottom line

The code is correct and consistent with the canonical allow-list. The production divergence is a data condition: five canonical capability rows are either inactive or absent on the production database. The `isActive: true` clause at `enterprise.service.ts:438` is what filters them out of the `/enterprise/options` response, and the empty-bucket elision in `groupCapabilities()` at `capability-groups.ts:170-185` is what makes the Workspace Access fieldset vanish from the User form. Reactivation via the Capabilities admin page restores the expected behaviour without any code change.